10｜安全基石（下）：Checkpointing，获得让AI“时光倒流”的超能力

Tony Bai · AI 原生开发工作流实战



你好，我是 Tony Bai。

在上一讲，我们深入了 Claude Code 的权限体系和沙箱机制，学会了如何为 AI 戴上一副坚固的“安全镣铐”。我们通过精细的规则，确保了 AI 在**行动之前**是可控的、可预测的。

这解决了一个核心的信任问题：“我如何确保 AI 不会做坏事？”。但随之而来的是一个更微妙、也更常见的困境。想象一下这个场景：你正在让 AI 帮你重构一个祖传的、长达 500 行的 Go 函数。

1. 你下达了第一个指令：“先将这个函数拆分成三个更小的私有函数。”AI 完成得很漂亮，你很满意，批准了它的文件修改。
2. 你信心大增，下达了第二个指令：“很好。现在，将这几个函数的核心逻辑，用 Go 的泛型进行重写，以提高复用性。”

3. AI 再次动手，修改了多个文件，引入了新的类型参数。但这次，你看着生成的新代码，眉头紧锁。它虽然能跑，但设计得过于复杂，可读性反而下降了。
4. 你想反悔，回到上一步的状态。但 AI 已经修改了 3 个文件，共计 15 处地方。你是要手动 `Ctrl+Z` 15 次，还是用 `git restore` 小心翼翼地恢复部分文件？无论哪种，都很痛苦，而且很容易出错。

这就是我们所说的“重构的恐惧”：当 AI 的修改偏离了我们的轨道时，我们缺乏一个简单、可靠、能够一键“反悔”的机制。

今天这一讲，我们就要彻底终结这种恐惧。我们将深入 Claude Code 的另一个独门绝技——**Checkpointing（检查点）**机制。你将学会如何获得让 AI“时光倒流”的超能力，为所有大胆的 AI 实验，备上一剂强大的“后悔药”。

Checkpointing 工作原理：会话级的“Git 快照”

Checkpointing 的理念，对于爱玩游戏的小伙伴儿来说，可能会非常亲切。它就像是游戏中的**自动存档（Auto-Save）**机制。

在你即将进入一场艰难的 Boss 战（一次复杂的 AI 操作）之前，游戏会自动为你创建一个存档点。如果你在战斗中失败了，你可以轻松地“读档”，回到进入战斗前的那个完美状态，然后换个策略再试一次。

Claude Code 的 Checkpointing 机制，就是你开发过程中的“自动存档系统”。它的工作原理可以概括为以下几点：

1. **触发时机：**每当你（用户）提交一个新的 Prompt 时，Checkpointing 系统就会被触发。它会在 AI 开始思考和行动之前，悄悄地为你的项目状态创建一个快照。
2. **保存内容：**这个快照，并不仅仅是代码的备份。它完整地记录了那一刻的“时空切片”，主要包含两部分信息：

   代码状态：当前会话中所有被 AI“染指”过的文件（即被读取或修改过的文件）的完整内容。

对话状态：到那一刻为止的完整对话历史。

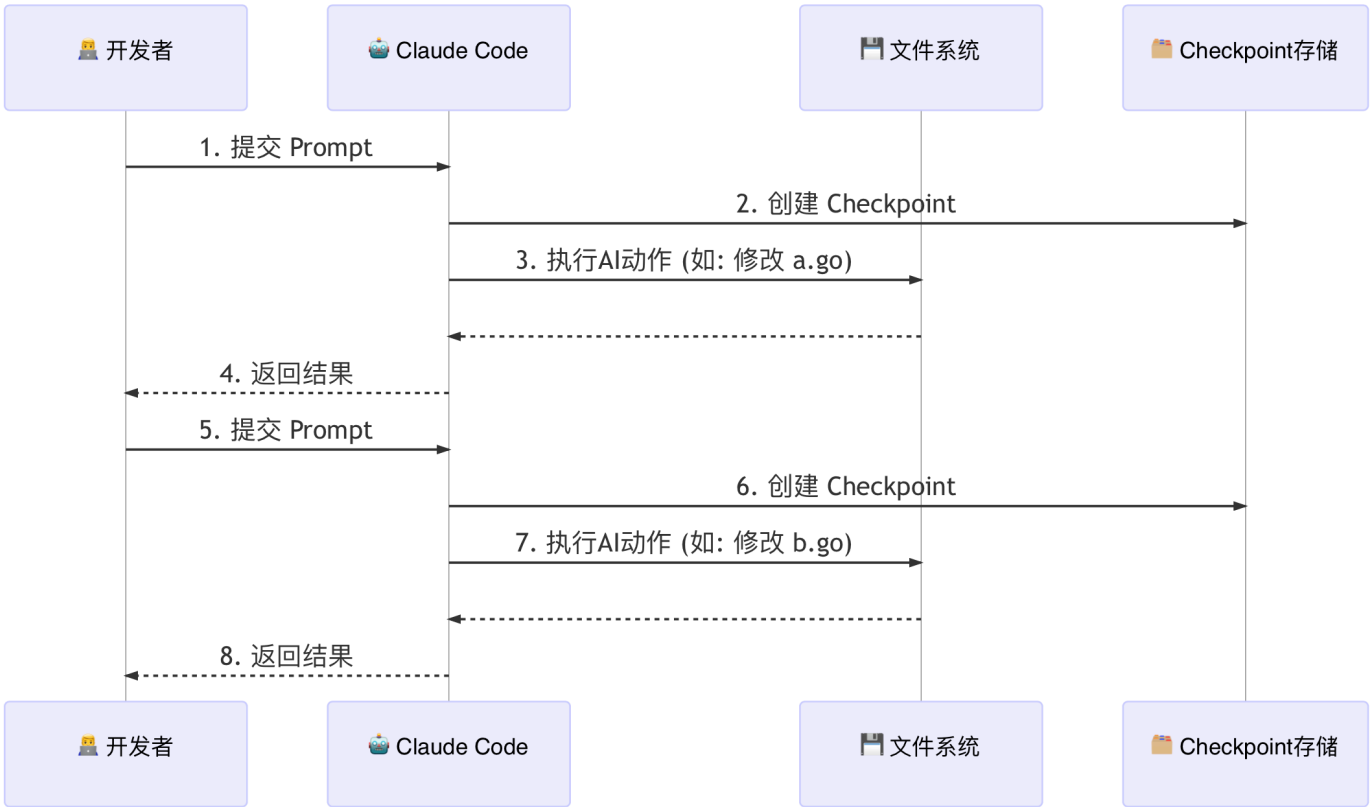
3. 为了更好地理解，你可以把它想象成一个由 Claude Code 在后台为你维护的、**与你项目主仓库完全隔离的“影子 Git 仓库”**。

   每当你提交 Prompt，Claude Code 就在这个“影子仓库”里帮你执行了一次 `git commit -a -m "Checkpoint before prompt N"`。

   这个操作极其轻量，并且绝对不会污染你项目本身的 `.git` 历史。它所有的历史记录都存放在 Claude Code 的内部目录中（如 `~/.claude/ file-history/`）。

这个机制的精妙之处在于，它将你的对话历史和代码变更历史，在每一个关键节点上进行了精确的绑定。

我们可以用一张图来清晰地看到这个流程：



看，在 AI 对你的文件系统进行任何“写”操作之前，一个包含了“之前”所有状态的完美快照，已经被安全地存盘了。这，就是我们能够“时光倒流”的底气所在。

实战：使用 `/rewind` 在任意检查点间穿梭

理论已经清晰，现在让我们进入最激动人心的实战环节。我们将亲历一次从“大胆重构”到“从容反悔”的全过程。

场景：假设我们正在维护一个 Go 函数，它根据不同的条件生成问候语。目前的代码充满了丑陋的 `if-else` 嵌套，可读性极差。

初始代码 `greeting.go`

复制代码

```go
package greeting

import "fmt"

// GetGreeting generates a greeting message.
// isFormal: whether to use a formal tone.
// isMorning: whether it is morning.
// isEvening: whether it is evening.
func GetGreeting(name string, isFormal bool, isMorning bool, isEvening bool) stri
    if isFormal {
        if isMorning {
            return fmt.Sprintf("Good morning, %s.", name)
        } else {
            if isEvening {
                return fmt.Sprintf("Good evening, %s.", name)
            } else {
                return fmt.Sprintf("Hello, %s.", name)
            }
        }
    } else {
        if isMorning {
            return fmt.Sprintf("Hey %s, good morning!", name)
        } else {
            if isEvening {
                return fmt.Sprintf("Hey %s, good evening!", name)
            } else {
                return fmt.Sprintf("Hey, %s!", name)
            }
```

```
        }
    }
}
```

第一步：第一次重构——走向清晰

我们向 Claude Code 下达第一个重构指令：

> `@greeting.go` 这个函数的 if-else 嵌套太深了，请帮我重构它，让逻辑更清晰。
> AI Agent 分析后，可能会提议使用 `switch` 语句，并给出了修改方案。我们觉得很棒，**批准**了这次修改。

代码状态 V2 (第一次重构后):

复制代码

```go
package greeting

import "fmt"

func GetGreeting(name string, isFormal bool, isMorning bool, isEvening bool) stri
    switch {
    case isFormal && isMorning:
        return fmt.Sprintf("Good morning, %s.", name)
    case isFormal && isEvening:
        return fmt.Sprintf("Good evening, %s.", name)
    case isFormal:
        return fmt.Sprintf("Hello, %s.", name)
    case !isFormal && isMorning:
        return fmt.Sprintf("Hey %s, good morning!", name)
    case !isFormal && isEvening:
        return fmt.Sprintf("Hey %s, good evening!", name)
    default:
        return fmt.Sprintf("Hey, %s!", name)
    }
}
```

这个版本比原来好多了。我们很满意。

第二步：第二次重构——一次“失败”的尝试

现在，我们想更进一步。我们觉得布尔参数太多了，想用一个枚举类型来代替 `isMorning` 和 `isEvening`。

我们下达了第二个指令：

> 很好。现在，请再次重构这个函数。定义一个名为 `TimeOfDay` 的枚举类型（可以是 `int` 常量），包含 `Morning`, `Afternoon`, `Evening`。让函数接收这个枚举类型，而不是两个布尔值。

AI 再次动手，给出了一套新的实现方案，包括了新的类型定义和修改后的函数。但我们看了一下，觉得它引入的 `iota` 和常量组让代码反而变得不那么直观了，而且它把 `isFormal` 的逻辑和时间逻辑混在了一起，可读性不升反降。

但是，我们“手滑”**批准**了这次修改。

代码状态 V3（第二次重构后，我们不满意的版本）：

复制代码

```go
package greeting

import "fmt"

type TimeOfDay int

const (
    Morning TimeOfDay = iota
    Afternoon
    Evening
)

func GetGreeting(name string, isFormal bool, time TimeOfDay) string {
    // ... 一段我们不满意的、复杂的实现 ...
}
```

项目现在处于一个我们不想要的状态。怎么办?

第三步：启动“时光机”——`/rewind`

现在，是时候召唤“后悔药”了。在 Claude Code 的输入框中，输入 `/rewind`（或者直接连按两次 `Esc` 键），“时光机”界面就会出现。

你会看到一个按时间顺序排列的**检查点列表**（旧的在上面，新的在下面），每一个都对应你提交的一次 Prompt：

```
Rewind

Restore the code and/or conversation to the point before…

  @greeting.go 这个函数的if-else嵌套太深了，请帮我重构它，让逻辑更清晰。
  greeting.go +17 -16

  请把if语句全部纳入switch语句中
  greeting.go +10 -18

  很好。现在，请再次重构这个函数。定义一个名为`TimeOfDay`的枚举类型（可以是`int`常量），包含`Morning`，`Afternoon`，
  greeting.go +15 -7

› (current)
```

更重要的是，当你通过方向键选择你要回退到的检查点并按下回车后，你会看到三个强大的“倒流”选项：

```
Rewind

Confirm you want to restore to the point before you sent this message:

  很好。现在，请再次重构这个函数。定义一个名为`TimeOfDay`的枚举类型（可以是`int`常量），包含`Morning`，`Afternoon`，
  `Evening`。让函数接收这个枚举类型，而不是两个布尔值。
  (2m ago)

The conversation will be forked.
The code will be restored +7 -14 in greeting.go.

› 1. Restore code and conversation
  2. Restore conversation
  3. Restore code

⚠ Rewinding does not affect files edited manually or via bash.
```

Rewind code（只回退代码）：

> **作用：**将你的文件系统恢复到选定检查点的状态，但保留完整的对话历史。

心智模型：“AI，我们刚才的讨论都很有价值，但我对你的具体实现不满意。让我们回到你动手之前，然后基于我们刚才的讨论，你再试一次。”

适用场景：AI 的思路正确，但代码实现有误。

Rewind conversation（只回退对话）：

作用：将你的对话历史回退到选定检查点，但保留文件系统的所有修改。

心智模型：“AI，你刚才做的代码修改是对的，但我发现我给你的指令说错了。让我们保留现在的代码，然后从我上一个指令开始，我换个说法。”

适用场景：代码修改符合预期，但你想从一个早期的对话节点，探索另一个完全不同的方向。

Rewind code and conversation（全部回退）：

作用：彻底的“时光倒流”。将文件系统和对话历史，都完美地恢复到选定检查点的状态。

心智模型：“AI，我们刚才那段探索完全是条死胡同。让我们假装它从未发生过，从这个更早的时间点重新开始。”

适用场景：当你对 AI 的一系列修改和你们的整个讨论方向都不满意时，这是最彻底、最干净的“重置”按钮。

在我们的场景中，我们对第二次重构的整个方向（引入枚举）都不满意，所以我们选择 Rewind code and conversation，然后按下回车。

瞬间，奇迹发生了：

1. 你的 `greeting.go` 文件内容，被完美地恢复到了我们满意的“代码状态 V2”。
2. 你的 Claude Code 对话界面，也回滚到了你下达第二次重构指令之前的状态。

一切都回到了那个“美好的时刻”。现在，你可以重新思考，然后给 AI 一个全新的、更好的指令。

Checkpointing 的边界：它不能做什么？

Checkpointing 虽然强大，但它不是万能的。清晰地理解它的边界，能帮助我们避免错误的使用，并将其与我们已有的工具（如 Git）进行更好的配合。

1. ** 它不跟踪 `Bash` 命令的副作用：** 这是最重要，也是最容易被误解的限制。Checkpointing 系统只快照文件内容的状态。对于通过 `!` 或 `Bash` 工具执行的命令所产生的副作用，它无能为力。例如，如果 AI 执行了：
    1. `! rm sensitive-data.log`
    2. `! git push --force`
    3. `! docker stop my-container`

这些操作是不可逆的。`/rewind` 无法帮你恢复被删除的文件、撤销强制推送，或重启被停止的容器。 这也是为什么我们在上一讲强调，对高风险的 `Bash` 命令，必须使用最严格的 `ask` 或 `deny` 权限规则。

2. **它不跟踪外部编辑：**Checkpointing 的“眼睛”只盯着由 Claude Code 的内置编辑工具（如 `Write`, `Edit`）所修改的文件。如果你在与 AI 对话的同时，自己在 VS Code 或 VIM 里手动修改了 `greeting.go`，这个手动的修改不会被记录在 Checkpointing 的历史中。

这通常不是问题，但需要记住：`/rewind` 恢复的是上一个检查点时刻、由 Claude Code 所知晓的文件状态。

3. **它不是 Git 的替代品：**这个误解必须澄清。Checkpointing 和 Git 是解决两种不同问题的工具，它们是互补而非替代关系。

维度	Checkpointing	Git
目的	**会话内、短期的、探索性的**状态恢复	**项目级、长期的、协作性的**版本历史管理
粒度	与**对话**绑定，粒度是“一次Prompt”	与**代码逻辑**绑定，粒度是“一次有意义的变更”
生命周期	**临时的**，随会话历史一起被清理（默认30天）	**永久的**，是项目的核心资产
协作	个人使用	**团队**协作的核心

极客时间

最佳实践是：在一个功能开发的过程中，你可以自由地使用 Checkpointing 进行各种大胆的尝试和回滚。当你完成了一个有意义的、稳定的功能点时，再使用 `git commit`，将这个阶段性的成果，永久地记录到项目的“正式历史”中去。

本讲小结

今天，我们为我们的 AI 原生开发工作流，装上了最后一块、也是最关键的一块安全拼图。我们获得了在 AI 世界里“反悔”的权力。

首先，我们直面了与 AI 进行复杂协作时的“**重构的恐惧**”，并明确了 Checkpointing 机制就是为了解决这个问题而生的“后悔药”。接着，我们深入了 Checkpointing 的工作原理，理解了它是一个由用户 Prompt 触发的、会话级的“Git 快照”系统，能够同时保存代码和对话的状态。

然后，通过一个完整的 Go 代码重构实战，我们亲手掌握了 `/rewind` 这一“时光机”指令，并深刻理解了其**三种不同回退模式**（只回退代码、只回退对话、全部回退）的精髓和适用场景。最后，我们清晰地划定了 Checkpointing 的边界，明确了它不能跟踪 `Bash` 命令的副作用，也不能替代 Git 作为正式的版本控制工具。

掌握了 Checkpointing，你就拥有了驾驭 AI 进行更大规模、更具探索性任务的信心。从“只敢让 AI 写点小函数”，到“敢于让 AI 重构整个模块”，这正是从 AI 使用者到 AI 驾驭者的关键一

跃。至此，我们的“安全基石”已经完全筑牢。我们学会了如何事前约束（权限与沙箱）和事后补救（Checkpointing）。现在，AI Agent 对我们来说，已经是一个足够安全的伙伴了。

那么，我们是否可以让它变得更“主动”一些呢？除了我们下达指令，它能否在某些特定时机，自动地为我们做一些事情？比如，每次修改完代码后，自动运行 `gofmt`？这就是我们下一讲要探讨的主题：**自动化之触——Hooks 机制**。我们将学习如何 AI 的生命周期中埋下“触发器”，实现更高阶的自动化。

思考题

除了今天我们演示的“代码重构”场景，请你再设想至少两个你认为 Checkpointing 机制能发挥巨大价值的日常开发场景。例如，探索一个新库的不同 API 用法？或者让 AI 生成多个不同风格的 UI 组件代码以供挑选？

请描述你的场景，并说明在这种场景下，你可能会更倾向于使用 `/rewind` 的哪种模式（只回退代码、只回退对话，或全部回退），以及为什么。欢迎在评论区分享你的创意工作流！如果你觉得今天的内容对你有帮助，也欢迎你分享给需要的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (11)

最新　精选



Biluota
2025-12-10 来自陕西

我目前能想到的场景是：
　　在进行前端网页开发时候，先通过一种想法让Claude生成一个效果页面，如果不错想保留，但是又想看看另一个想法的效果，就进行 `Rewind conversation` 会话回退。然后提出另一个效果需求进行实现，再对比需要保留哪种。
但是上面的示例到最后就会有个疑问，这种流程下开发到最后，如果对比之后，想保留最新的一版，其他各对比效果代码想删除，怎么实现？能否通过 Checkpointing 实现某一个实现效果的保留而删除其他代码，还是需要手动删除其他代码。

还有就是关于只回退对话的情况，还有哪些场景下使用非常合适，以及怎么使用，老师能否多举几个例子开下脑洞。

作者回复: 1. 关于“多版本对比后，如何保留/删除”的问题

Checkpointing 本质上是一个线性的时光机，它不具备“分支管理”的能力。

这样一来，当你用 Rewind Conversation（保留代码，回滚对话）做了 A 方案，然后又做了 B 方案，此时你的代码库其实是“A + B”的混合状态（如果文件不冲突），或者是 B 覆盖了 A（如果修改同名文件）。

那怎么解决呢？ 不要只依赖 Checkpoint，要结合 Git！

可以采用Git Worktree/Branch的方案： 在探索 A 方案前，先 git checkout -b feat/option-a。做完觉得不错，提交 commit。然后切回主分支，再开 feat/option-b 做 B 方案。

Checkpoint 的角色是在同一个方案内部试错用的（比如 A 方案写了一半歪了，回滚重写）。而不同方案的对比，应该交给 Git 分支来管理。这样
你最后只需 Merge 你想要的那个分支即可，无需手动删除废代码。

2. 关于 Rewind Conversation Only（只回退对话）的更多脑洞场景

这个功能的精髓在于：“代码是对的，但我想换个聊法”。

随便举一个场景： 连续的微调指令（避免上下文污染）

你让 AI 生成了一个复杂的 SQL 查询（代码生成了，是对的）。

你回滚的意图是你不想让这个冗长的 SQL 生成过程占据上下文窗口。

执行 Rewind Conversation。代码保留在文件里，但对话历史被清空了。你接着说：“基于刚才生成的 SQL 文件，帮我写一个 Go 函数来调用它。”
这样可以节省 Token，保持上下文纯净。

不知道上述场景和说明是否能帮你打开思路:)

最后记住，Checkpointing 是战术级的“撤销”，Git 才是战略级的“版本控制”。两者结合起来用，基本可以覆盖大部分场景。

共 4 条评论>

23



Biluota

2025-12-10 来自陕西

好像/clear后，之前的Checkpointing也会消失，这块也需要注意，一个需求完全完成后再/clear

作者回复: 非常敏锐的观察！确实如此。

Checkpointing 是绑定在当前 Session（会话） 的。当你执行 /clear 时，本质上是开启了一个新的会话上下文。

虽然文件系统的修改保留了，但新会话丢失了之前会话的“时光机”索引。你无法再用 /rewind 回退到 /clear 之前的某个中间状态。

正如你所说，“一个需求（Task）对应一个 Session”。只有当当前功能完全开发完毕、测试通过并提交（git commit）后，再执行 /clear 清理战场
，开启下一个任务。这样既保持了上下文的纯净，又规避了回滚链断裂的风险。

共 2 条评论>

13



极客酱酱

2025-12-10 来自四川

全链路环境升级与迁移测试：当需要升级核心依赖（如从 Vue 2 升 Vue 3，Webpack 转 Vite，或 Python 3.8 升 3.12）时，利用 Checkpointing 进行“迁移探针”测试，看看有哪些 Breaking Changes。

作者回复: 👍

4

Teresa

2025-12-16 来自河北

我有一个问题，就是现在大公司的开发项目是不是大都没使用AI进行编程？当要对一个已有的开发项目进行全面总结的时候，是不是的所有代码是会被当作提示词送到提供AI的API接口里？这样是不是也会造成公司项目的代码泄露呢？尤其一些核心代码。如果考虑这方面的安全，有办法解决吗？

作者回复: 这的确是一个非常现实且关键的安全问题。

并且，这种风险是实际存在的：当你使用公共云端 AI，你的代码确实会作为 Prompt 发送到云端。如果服务商（如 OpenAI）将这些数据用于训练，
那就存在泄露核心代码的风险。

不过经过这几年的发展，大公司应该也有了成熟的解决方案，并且大公司拥抱ai编程一点不落后，你关注一下腾讯、阿里、字节的技术公众号就能了解一般。

比如，大公司通常购买的是企业版 API（Enterprise License）。这种协议明确规定：云厂商不保留、不查看、不用客户数据训练模型。数据对于云
厂商来说是“过路即焚”的。这是法律层面的保障。

再比如，大公司财大气粗，可能本身也有自己的大模型，或者有能力运行大规模的模型集群，这样就可以做私有化部署大模型。国内开源大模型能力
也处于头部，私有化部署后，一般也可以满足日常编码需求。

共 2 条评论>

3



hunter

2026-02-03 来自浙江

vs code claude code插件，没有/rewind这个命令

作者回复: Checkpointing (/rewind) 应该是 Claude Code CLI (终端版) 的独有功能，它依赖于 CLI 对本地文件系统的深度控制和“影子 Git”机制。VS Code 插件运行在受限的沙箱中，暂不支持此特性。

1



Ryan

2026-01-29 来自上海

月光宝盒之般若波罗蜜

作者回复: 到位 👍

1

龍蝦
2026-01-22 来自浙江

checkpointing 只能向后回退，回退后后悔了，想向前该怎么办呢?

> 作者回复: 很可惜，目前的 Checkpointing 机制是“单向”的，即回退即销毁： 当你执行 /rewind 回退到某个旧节点并开始新的操作（比如输入新 Prompt）时
> ，那个节点之后的“未来”分支就会被永久丢弃。
>
> 如果非要向前，只能依靠 Git的“外部记忆”。
>
> 在你准备进行大范围 /rewind 之前，如果你觉得现在的状态可能还有用，先切出去手动 git commit 或 其他命令 保存一下。这样即使 Checkpoint 丢了，Git 里还有备份。不过，上下文是救不回来了。

1

夏天
2026-06-02 来自北京

回溯菜单列出了您在会话期间发送的每个提示。选择您想要操作的点，然后选择一个操作：
恢复代码和对话：将代码和对话都恢复到该点
恢复对话：回溯到该消息，同时保持当前代码
恢复代码：恢复文件更改，同时保持对话
从此处总结：将此点之后的对话压缩为摘要，释放 context window 空间
到此处总结：将此点之前的对话压缩为摘要，保持后续消息完整
算了：返回消息列表而不做任何更改

https://code.claude.com/docs/zh-CN/checkpointing
老师，现在增加了三项内容

> 作者回复: 👍

---

open！？
2026-04-30 来自浙江

现在是三个不同的：
› 1. Restore conversation
2. Summarize from here

3. Never mind

作者回复: 👍 。cc一直在演进，并增加新特性。也欢迎各位小伙伴儿多多留言，将新的特性或类似新增的选项补充进来。




子衿

2025-12-10 来自浙江

来啦

作者回复: [手动抱拳]